ARQUITECTURA DE COMPUTADORES

GRADO EN INGENIERÍA INFORMÁTICA – INGENIERÍA DEL SOFTWARE

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

DEPARTAMENTO DE ARQUITECTURA Y TECNOLOGÍA DE COMPUTADORES

UNIVERSIDAD DE SEVILLA

http://www.atc.us.es

INDICE

- Tecnologías de memoria
- Jerarquía de memoria
- Memorias caché

MATERIAL DE LECTURA DEL TEMA

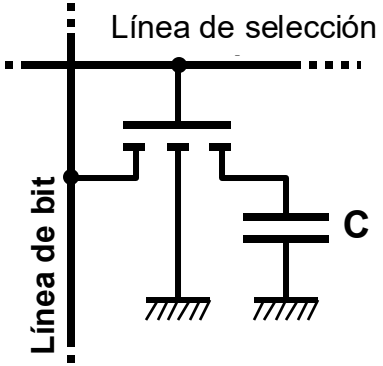
- REFERENCIAS INCLUIDAS EN CADA DIAPOSITIVA:
  - En algunas diapositivas se hace referencia (en la esquina superior izquierda) al material adicional de clase que completa la explicación de algunos conceptos.
- LECTURAS OBLIGATORIAS PARA EL TEMA COMPLETO (PARTES I y II):
  - "Computer organization and design: the hardware-software interface. RISC-V edition" David A. Patterson, John L. Hennessy
    - Del capítulo 5:
      - Apartados 5.1, 5.3, 5.4 y 5.8
      - Apartado 5.7
  - Como alternativa, puede consultarse el siguiente libro: "Estructura y Diseño de Computadores" David A. Patterson, John L. Hennessy
    - Del capítulo 5:
      - Apartados 5.1, 5.2, 5.3 y 5.5 (Jerarquía de memoria y Memoria Caché)
      - Apartado 5.4 (Memoria Virtual)
  - W. Stallings, "Organización y Arquitectura de Computadores" (7ª edición):
    - Del capítulo 5, apartado 5.1: Memoria interna (Memoria principal semiconductora).
- BIBLIOGRAFÍA DE CONSULTA DEL TEMA:
  - Bruce Jacob, Spencer W. Ng, David T. Wang, "Memory Systems. Cache, DRAM, Disk", Morgan Kaufmann Publishers, 2008.

Tecnologías de memoria semiconductora

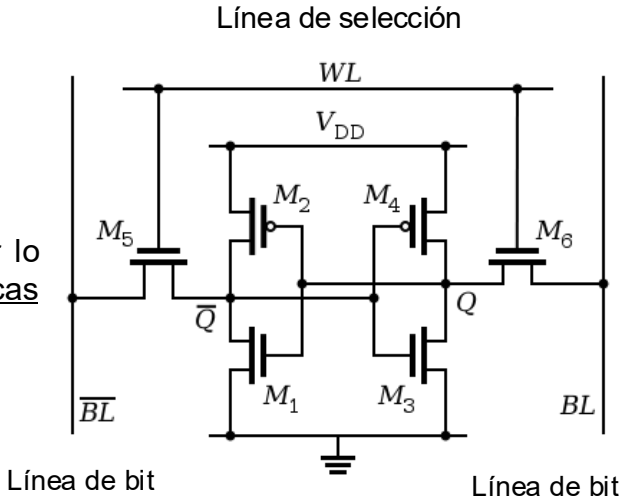
- El elemento básico de una memoria semiconductora es la celda:
  - Presenta 2 estados estables que se pueden usar para representar el 1 y el 0.
  - Puede escribirse en ellas para fijar su estado.
  - Puede leerse para detectar su estado.

- Dos categorías principales:
  - **ROM** (*read-only memory*):
    - Memorias no volátiles.
    - Los datos se almacenan de forma permanente o semipermanente (ej. de uso: programa de la BIOS, firmware de dispositivos de E/S, programas de los microcontroladores, discos SSD, memoria USB, etc.)
  - **RAM** (*random-access memory*):
    - Memorias volátiles.
    - Capacidad de lectura y escritura.
    - Dos tipos: **SRAM y DRAM**.

- **RAM dinámica (DRAM)**
  - Celda de memoria basada en CONDENSADORES (dispositivo analógico).
    - El condensador se carga de forma que mantiene una tensión que es interpretada como un nivel lógico (0 o 1).
    - Periódicamente, el contenido de la celda se lee y se vuelve a escribir (esta operación se llama refresco). Esto evita que el condensador se descargue y, por tanto, que el contenido de memoria se pierda.
  - Usada para memorias grandes: Memoria principal.

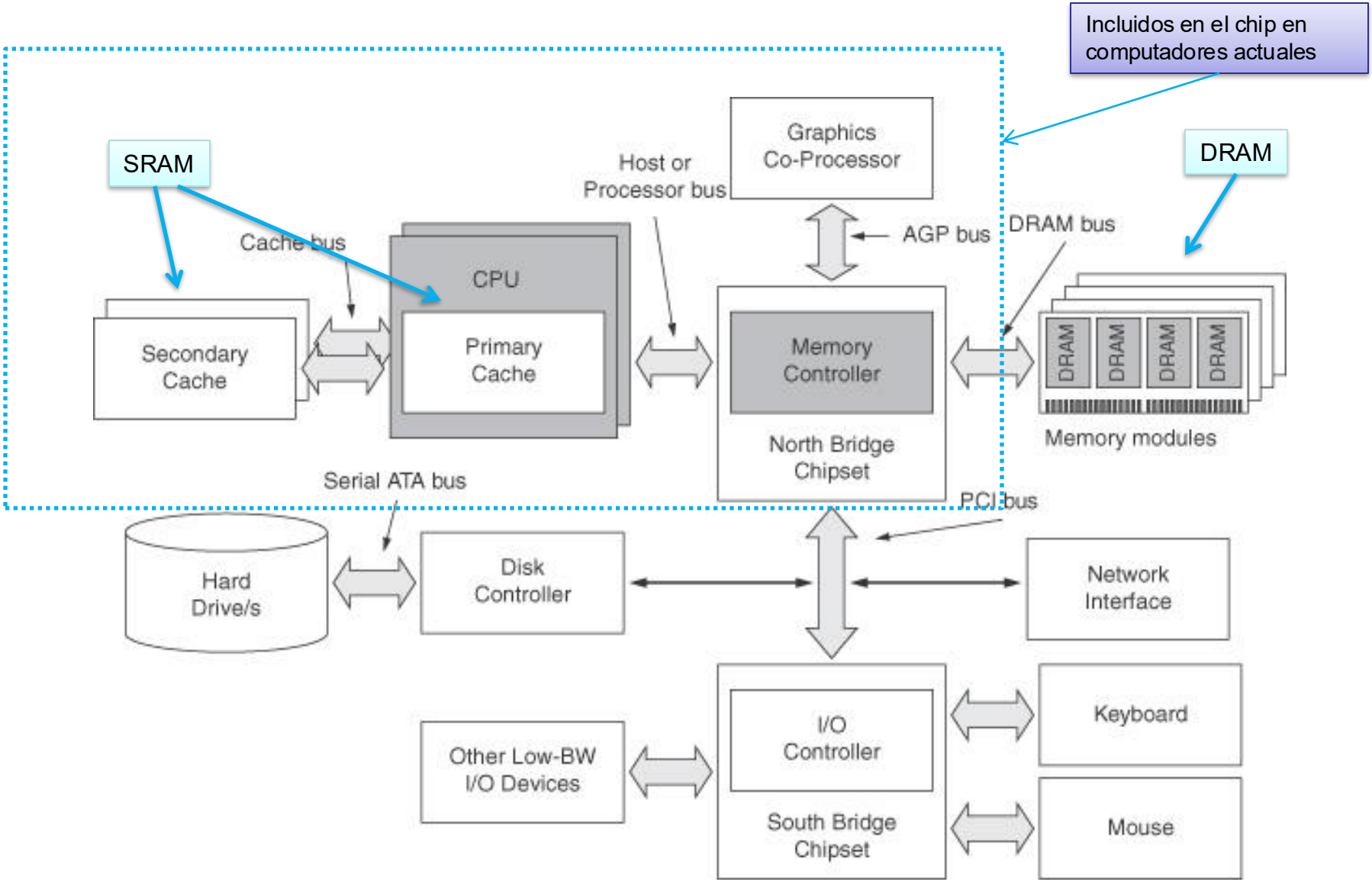


- **RAM estática (SRAM)**
  - Celda de memoria basada en BIESTABLES.
    - No requiere “refresco”.
    - Dispositivo digital
  - Usada para memorias caché.



- **Comparación:**
  - DRAM <u>más densas</u> (las celdas son más simples y más pequeñas, por lo tanto, caben más celdas por unidad de superficie) y <u>más económicas</u> (menor coste por bit) que SRAM.
  - SRAM más rápida que DRAM

Tecnologías de memoria semiconductora
Ubicación de la memoria en un computador



Jerarquía de memoria
Introducción

- Tecnología: existen diferentes tipos de memoria según criterios de tamaño, velocidad y coste:
  - Las más rápidas son más caras (SRAM)
  - Las más baratas y las de mayor capacidad son más lentas (DRAM, díscos magnéticos y SSD).
- Necesidad de grandes cantidades de memoria rápida con tiempos de acceso y transferencia pequeños, y de bajo coste.
- El objetivo de la jerarquía de memoria es dotar al computador de un sistema de memoria de coste y capacidad similar al de la memoria más barata y casi tan rápido como la memoria más rápida.
- Principio de localidad: los programas favorecen una parte de su espacio de direcciones.
  - Durante diferentes periodos de tiempo, los programas acceden con mayor probabilidad a diferentes subconjuntos de direcciones de memoria.
    - El código y los datos accedidos durante un periodo de tiempo determinado pueden estar almacenados temporalmente en una memoria rápida y pequeña.
  - Este principio hace posible que se alcance el objetivo de la jerarquía de memoria.

Idealmente sería deseable una capacidad indefinidamente grande de memoria tal que cualquier particular…palabra estuviese inmediatamente disponible… Estamos… forzados a reconocer la posibilidad de construir una ***jerarquía de memorias****, cada una de las cuales tenga mayor capacidad que la precedente pero que sea menos rápidamente accesible.*

A.W. Burks, H.H. Goldstine y J. Von Neumann,

Discusión preliminar del diseño lógico de un instrumento de cálculo electrónico (1946)

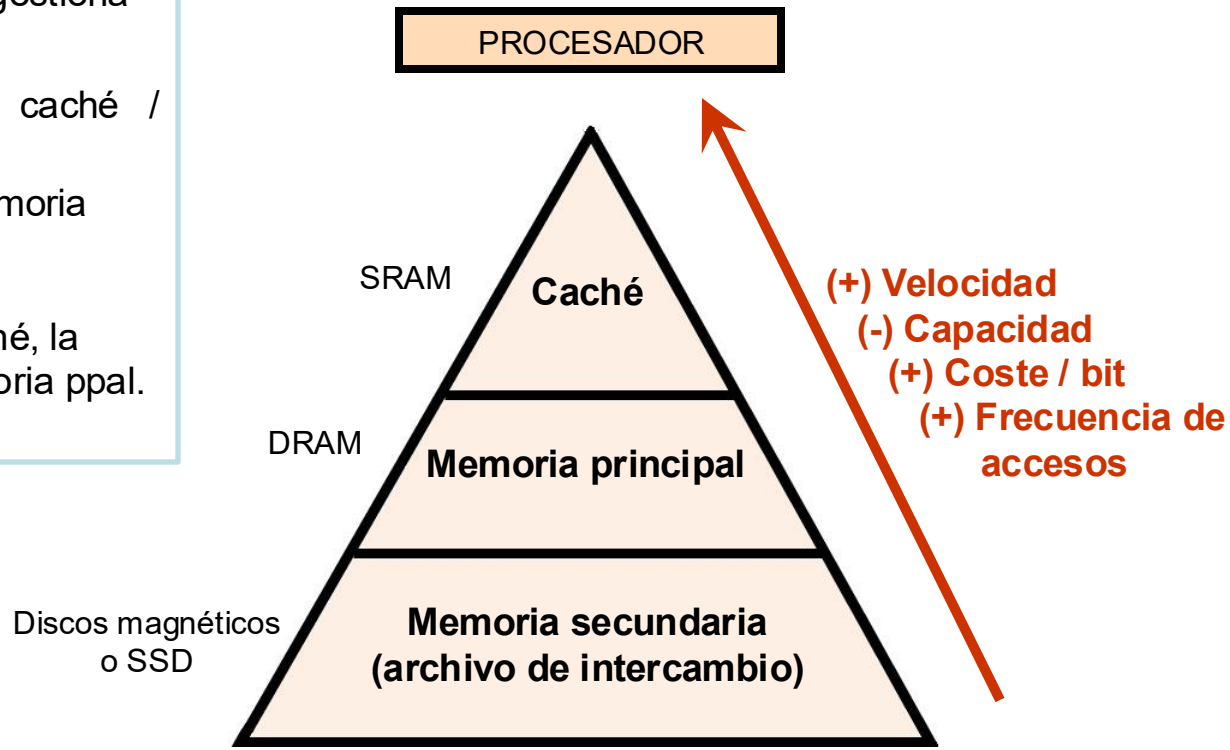
Jerarquía de memoria
Definición

- Una jerarquía de memoria está organizada en varios niveles, cada uno más pequeño, más rápido y más caro por *byte* que el inferior.
- De forma transparente al programador, el sistema se comporta como si fuera una única memoria cuya capacidad es tan grande como la del nivel más grande de la jerarquía y cuya velocidad se aproxima a la del nivel más rápido.

En cada momento, la jerarquía se gestiona entre dos niveles adyacentes:

- Jerarquía de memoria caché: caché / memoria principal
- Jerarquía de memoria virtual: memoria principal / memoria secundaria

Ejemplo: si un dato no está en caché, la transferencia se realiza entre memoria ppal. y caché.



En este tema se estudiará solamente un único nivel de caché; sin embargo, para conseguir mejorar la velocidad de acceso al sistema de memoria, los fabricantes han ido incrementando con el tiempo el número de niveles de caché (L1, L2 y L3)

Jerarquía de memoria
Valores Típicos

Jerarquía de Memoria Caché

Procesador

Nivel	1	2	3	4
Nombre	**Registros**	**M. Caché**	**M. Principal**	**M. Secundaria**
Tamaño	< 4 KB	32 KB a 8 MB	< 1 TB	> 1TB
Tecnología	Memoria CMOS de varios puertos	CMOS SRAM	CMOS DRAM	Disco Magnético o SSD
Tiempo Acceso* (ns)	0,1 a 0,2	0,5 a 10	30 a 150	5.000.000
Ancho de banda (MB/s)	1.000.000 a 10.000.000	20.000 a 50.000	10.000 a 30.000	100 a 1.000
Manejado por …	Compilador	Hardware	Sistema Operativo y hardware	Sistema Operativo

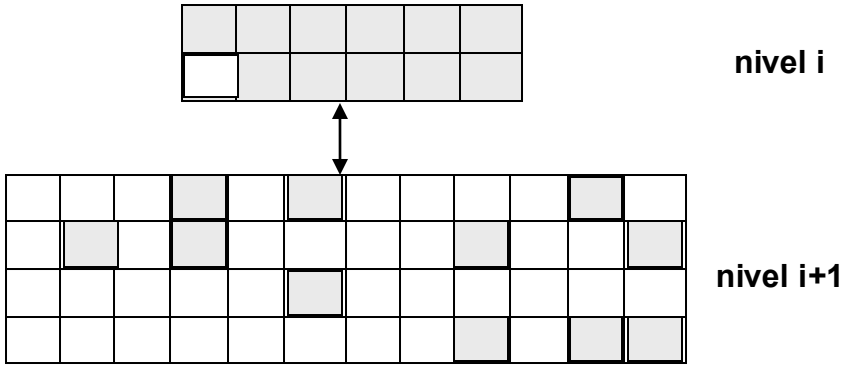
**Valores típicos en el año 2017*

Jerarquía de Memoria Virtual

Fuente: “Computer Architecture: A Quantitative Approach .” J. Hennessy, D. Patterson. 6th edition. 2017

Jerarquía de memoria
Propiedades

- **Inclusión**: Cualquier información almacenada en un nivel de memoria, debe encontrarse también en los niveles inferiores.



- **Coherencia**: Las copias de la misma información existentes en los distintos niveles deben ser consistentes: si se modifica la información en un nivel, dicha información debe actualizarse también en el resto de niveles.
  - Por motivos de eficiencia, se puede permitir la incoherencia en la información de forma temporal.

- **Localidad de referencia (principio de localidad)**: Los programas favorecen una parte de su espacio de direcciones en cualquier instante de tiempo.
  - **Localidad temporal**: *Si se referencia una dirección, esta tenderá a ser referenciada pronto* (p.e., bucles, etc.)
  - **Localidad espacial**: *Si se referencia una dirección, las direcciones cercanas a ella tenderán a ser referenciadas pronto* (p.e., vectores, secuenciamiento de instrucciones, etc.).

Jerarquía de memoria
Propiedades: Ejemplo de localidad de referencia

Ejemplo: Suponga que se ejecuta el código máquina correspondiente al siguiente fragmento de código C y que las variables *suma* e *i* se almacenan en memoria (es decir que el compilador no usa registros para contener sus valores):

```
…
suma = 0;
for (i=0; i<N; i++) {
    suma = suma + a[i] + b[i];
    a[i] = a[i] – 1;
}
…
```

Localidad temporal en datos:

- Se accede a *suma* e *i* en todas las iteraciones
- En cada iteración, se accede 3 veces a *a[i]*.

Localidad espacial en instrucciones:

- Las instrucciones del código máquina se encuentran en posiciones contiguas de memoria.

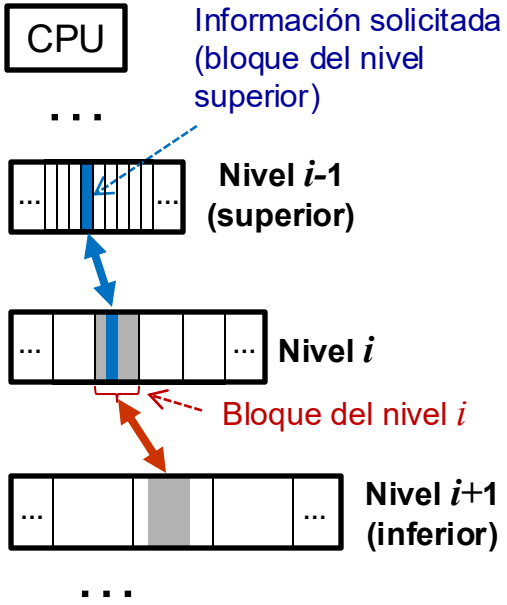
Localidad espacial en datos:

- Si *suma* e *i* son variables locales, se encuentran en posiciones de memoria cercanas dentro de la pila.
- Después de acceder a *a[i]*, se accederá en las siguientes iteraciones a los siguientes elementos del vector (que se encuentran en posiciones de memoria contiguas).
- Idem para *b[i]*.

Localidad temporal en instrucciones:

- En cada iteración, se accede de nuevo a todas las instrucciones del bucle.

- La CPU sólo accede al nivel más alto (el más rápido y de menor tamaño).
- Solo se realizan accesos entre niveles adyacentes:
  - Las transferencias entre niveles adyacentes se hacen con **bloques** de múltiples bytes.
  - El bloque es la unidad mínima de información que puede estar presente o no en un nivel dado.
  - El tamaño de los bloques depende de los dos niveles adyacentes que se gestionan. En las técnicas que estudiaremos, el tamaño de los bloques es fijo.
- Funcionamiento (ejemplo de lectura al nivel i de la jerarquía):
  - Cuando el nivel superior solicita la lectura de información al nivel i, se produce un acierto o un fallo:
    - **Acierto** (*hit*): la información se encuentra en uno de los bloques del nivel i y es transferida al nivel superior.
    - **Fallo** (*miss*): la información no se encuentra en ninguno de los bloques del nivel i. El bloque que contiene la información se transfiere del nivel inferior al nivel i y a continuación se transfiere la información al nivel superior.
  - En el primer nivel de la jerarquía, es la CPU la que solicita la lectura de una palabra (diremos que la CPU emite una dirección).



Jerarquía de memoria
Conceptos básicos (continuación)

- Para los accesos al nivel i se definen las siguientes medidas:
  - **Frecuencia de aciertos** (*hit rate, hr)*: fracción de accesos al nivel i que han acertado.
  - **Frecuencia de fallos** (*miss rate, mr*): fracción de accesos al nivel i que han fallado.
  - **Tiempo de acierto** (*hit time*): tiempo de acceso al nivel i en caso de acierto (incluye el tiempo necesario para determinar si el acceso es acierto o fallo).
  - **Penalización por fallo** (*miss penalty*): tiempo necesario para transferir un bloque (el que contiene la información buscada) desde el nivel inferior. Se puede dividir en dos componentes:
    - Tiempo necesario para iniciar la transferencia o sobrecarga del primer acceso (llamado, habitualmente, **tiempo búsqueda** o **tiempo de acceso**). Este tiempo es independiente del tamaño de bloque.
    - Tiempo requerido para transferir el bloque (llamado, habitualmente, **tiempo de transferencia**) .

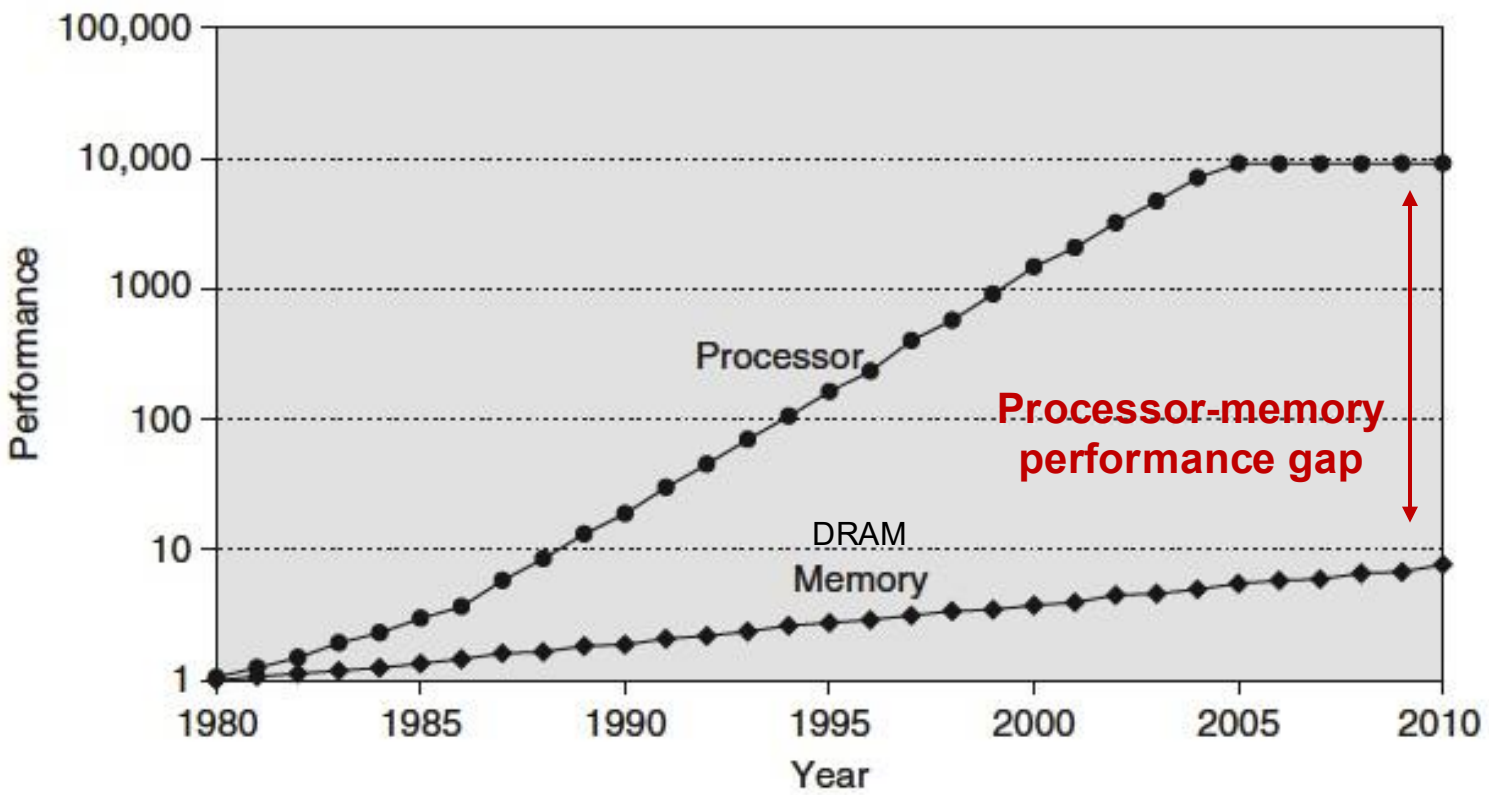
Jerarquía de memoria
Evaluación del rendimiento

- Tiempo medio de acceso a memoria *(AMAT, Average Memory Access Time)*

AMAT= Tiempo de acierto + Frec. de fallos × Penalización por fallo

- Permite evaluar el rendimiento de cualquier nivel de la jerarquía (o de la jerarquía completa)
- La optimización de un sistema se puede basar en la optimización de cualquiera de los tres parámetros del tiempo medio de acceso a memoria.
  - Dependen de las características del diseño de los niveles de la jerarquía.
  - Los tres parámetros están interrelacionados, los cambios en el diseño realizados para la optimización de cualquiera de ellos puede afectar negativamente al resto.
- La frecuencia de fallos depende también del programa en ejecución.

Introducción a las memorias caché

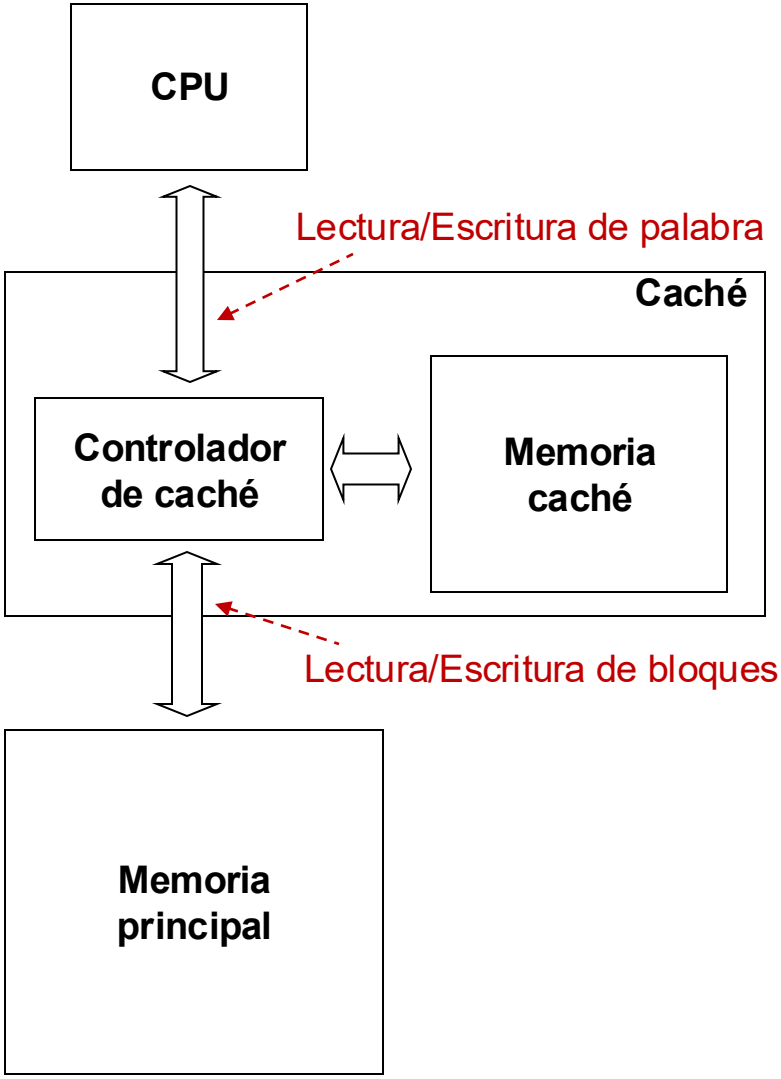


Fuente: Hennessy, J. L. and Patterson, D. A., *Computer Architecture. A quantitative Approach*

El rendimiento de los procesadores y memorias ha crecido de manera desigual, haciendo que la brecha existente haya ido aumentando con el tiempo. Esto ha provocado la necesidad de una memoria intermedia entre el procesador y la memoria principal: **memoria caché.**

Introducción a las memorias caché

- Representa el nivel de jerarquía de memoria entre la CPU y la memoria principal.
- Se aplican todos los conceptos vistos para la jerarquía de memoria.
  - Los bloques de caché se denominan **líneas**
- Acceso a memoria:
  1. La CPU emite una dirección de memoria principal (coloca la dirección en el bus de direcciones).
  2. El controlador de caché usa la dirección para comprobar si el bloque que contiene el dato se encuentra en caché.
  3. Si no se encuentra, se produce un fallo y el bloque es transferido desde la memoria principal a la caché.
  4. Si el bloque está en caché, se lee el dato y se envía a la CPU.



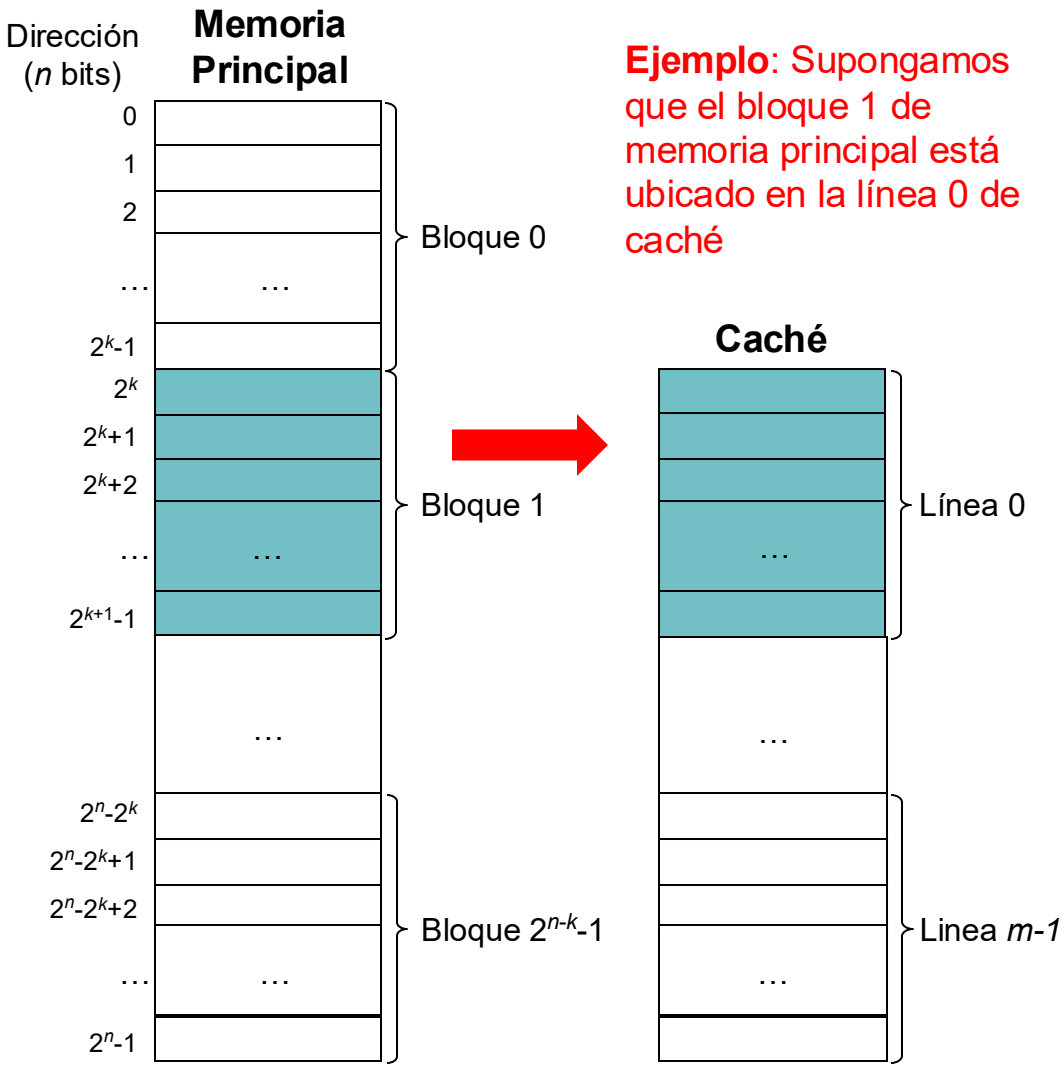
Tamaño de línea (bloque)	4 a 128 bytes
Tiempo de acierto	1 a 4 ciclos
Penalización de fallo:	8 a 32 ciclos
Tiempo de acceso	6 a 10 ciclos
Tiempo de transferencia	2 a 22 ciclos
Frecuencia de fallos	1% a 20%
Tamaño de caché	1 a 256 KB

Parámetros de las cachés (1990), extraído de Hennessy, J. L. and Patterson, D. A., *Computer Architecture. A quantitative Approach*

Estructura Memoria Caché/Memoria Principal

- Supongamos que las direcciones son de *n* bits.
- La memoria principal está constituida por 2^n bytes.
- Supongamos que cada bloque está constituido por 2^k bytes.
- La memoria principal está formada por 2^{n-k} bloques.
- Supongamos que la caché tiene *m* líneas (bloques).
- La dirección emitida por la CPU se descompone en:
  - **Dirección de bloque** (DB): permite direccionar los diferentes bloques de memoria.
  - **Desplazamiento** (D): permite direccionar los diferentes bytes del bloque requeridos por la CPU.

Dirección	DB	D
	n-k bits	*k* bits



Ejemplo: Supongamos que el bloque 1 de memoria principal está ubicado en la línea 0 de caché

Tipos de memoria caché

- Según su estructura interna y funcionamiento:
  - Ubicación e identificación de bloques
  - Política de reemplazo
  - Política de escritura

- Según el tipo de información que contienen:
  - Unificadas o separadas

Ubicación de bloque

- ¿En qué líneas de cachés puede ubicarse un determinado bloque de memoria?
  - **Cachés de correspondencia directa (o de mapeado directo)**: cada bloque tiene asociada una única línea en la que puede ubicarse.
    - Es necesario establecer una **función de correspondencia** entre bloques y líneas
    - La opción más simple es utilizar la función **módulo.**
      - El valor binario de N mod 2^k es igual a los k bits menos significativos de N.
      - Ejemplo: 9 mod 4 = 1 ➞ 1001

```
Línea-seleccionada = Dirección-de-bloque MOD Número-de-líneas
```

  - **Cachés totalmente asociativas (o completamente asociativas)**: los bloques pueden ubicarse en cualquier línea.
  - **Cachés asociativas por conjuntos de k vías**: cada bloque tiene asociado un conjunto de k líneas en las que puede ubicarse.
    - Conjunto: conjunto de líneas donde puede ubicarse un bloque. Es necesaria una **función de correspondencia** entre conjuntos y líneas.
    - Cada conjunto está formado por k líneas (cada una de ellas se denomina **vía**)
    - Una caché de 2^m líneas tiene 2^m / k conjuntos

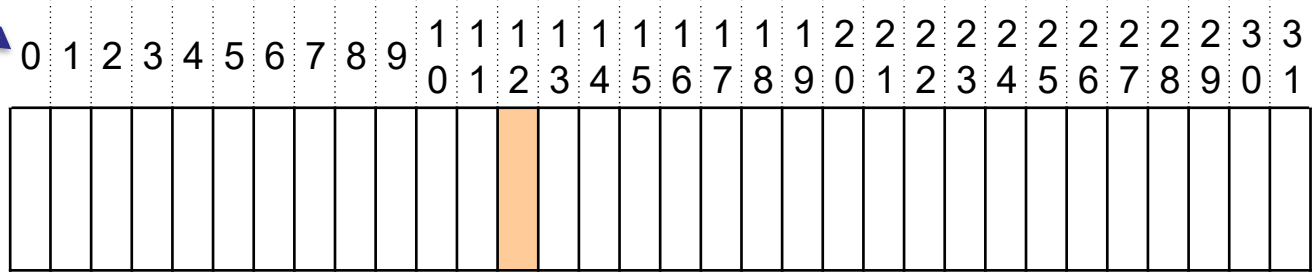
```
Conjunto-seleccionado = Dirección-de-bloque MOD Número-de-conjuntos
```

Ejemplo de ubicación de bloque (I)

¿En qué líneas de caché podemos ubicar el bloque de memoria 12; es decir, la dirección de bloque es 12 (en binario $1100_{(2}$)?

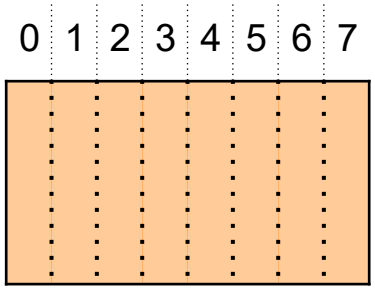
Dirección de bloque

Memoria principal de 32 bloques

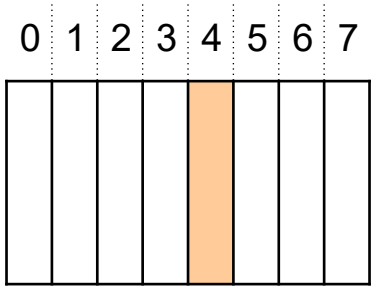


Memoria caché de 8 líneas

Completamente asociativa

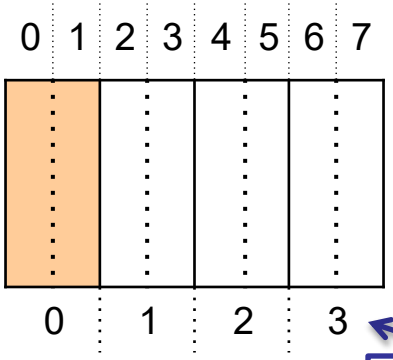


Correspondencia directa (8 líneas)



12 mod 8 = 4

Asociativa por conjuntos de 2 vías (4 conjuntos)



12 mod 4 = 0

4 conjuntos

En las cachés, el sombreado indica las líneas en las que se puede ubicar el bloque 12.

El grado de asociatividad es el número de líneas en las que se puede ubicar un bloque (el número de líneas sombreadas).

Equivale a los 3 bits menos significativos de $1100_{(2}$ → $\mathbf{100_{(2}}$

Equivale a los 2 bits menos significativos de $1100_{(2}$ → $\mathbf{00_{(2}}$

Ejemplo de ubicación de bloque (II)
Continuación del ejemplo anterior

Supongamos una memoria de 2KB (11 bits) y bloques de 64 bytes (por lo tanto, la memoria tiene 32 bloques de 64 bytes). Entonces la caché descompone las direcciones en:

11 bits	Dir. de bloque (5 bits)	Desplazamiento (6 bits)

En la caché de mapeado directo (inicialmente vacía) se producen los siguientes accesos:

Primer acceso: La CPU emite la dirección de memoria 0x33C = 011 0011 $1100_{(2}$

Dir. de bloque 12

Desplazamiento 60

El byte correspondiente a la dirección 0x33C se encuentra en la posición 60 del bloque 12

0x33C

01100	111100

El bloque 12 se ubica en la línea 4 (diapositiva anterior)

Como la línea está vacía, el bloque 12 se lee de memoria y se almacena en la línea 4.

Segundo acceso: La CPU emite la dirección de memoria 0x73A = 111 0011 $1010_{(2}$

Dir. de bloque 28

Desplazamiento 58

El byte correspondiente a la dirección 0x73A se encuentra en la posición 58 del bloque 28

0x73A

11100	111010

El bloque 28 también debe ubicarse en la línea 4

¿Cómo puede comprobar el controlador de caché si el bloque ubicado en la línea 4 es el bloque 28?

Si un bloque se ubica en la línea 4, entonces los 3 bits menos significativos de su dir. de bloque son **100**. Para identificar al bloque bastaría con **almacenar** junto a la línea **el resto de la dir. de bloque**.

Identificación de bloque

- ¿Cómo se sabe si un bloque de memoria está ubicado en caché a partir de la dirección de memoria emitida por la CPU?
  - Para poder identificar al bloque de memoria que está almacenado en cada línea, la caché almacena también la dirección de bloque (o parte de ella). Esta información se denomina **etiqueta**.
    - En las cachés **completamente asociativas**, puesto que un bloque puede estar en cualquier línea de caché, es necesario almacenar la dirección de bloque completa (es decir, la etiqueta es la dirección de bloque).

      La dirección de memoria emitida por la CPU se descompone en:

Etiqueta (dir. de bloque)	**Desplazamiento**

    - En las cachés de **correspondencia directa** y en las **asociativas por conjuntos:**
      - Los bits menos significativos de la dirección de bloque identifican a la línea o conjunto en que se encuentra el bloque. Esta información se denomina **índice**. El índice no se almacena con las líneas.
      - El resto de bits de la dirección de bloque se usan como **etiqueta** y se almacenan con las líneas (véase diapositiva anterior).

      La dirección de memoria emitida por la CPU se descompone en:

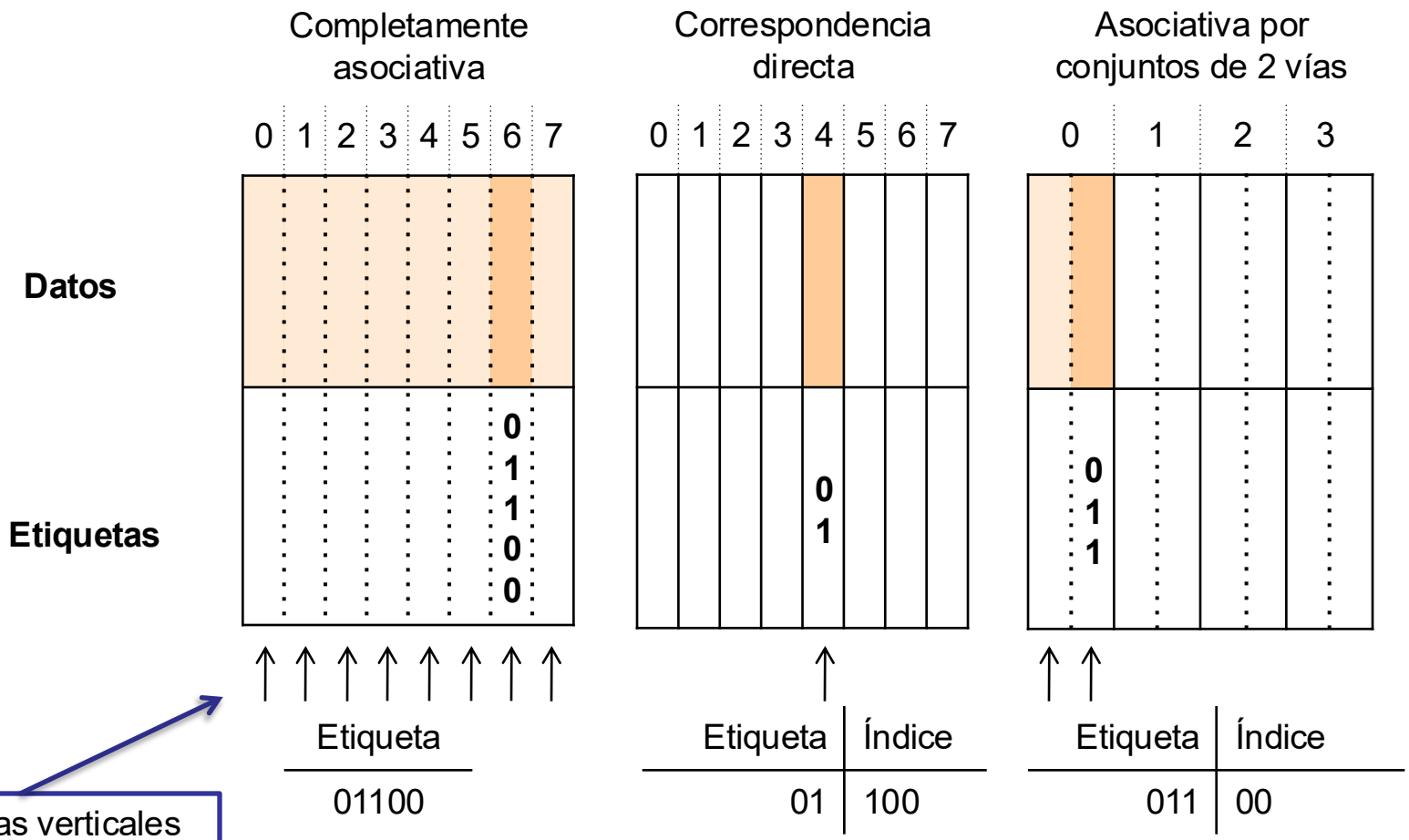
Etiqueta	**Índice**	**Desplazamiento**
Dirección de bloque		

Identificación de bloque

- El **índice** se usa para seleccionar el conjunto en el caso de las cachés asociativas por conjunto; o la línea, en las de correspondencia directa. No existe en las cachés completamente asociativas.
- Las **etiquetas** de las líneas que pueden contener el bloque buscado se comparan **en paralelo** con los bits correspondientes de la dirección de memoria indicada por la CPU (en las cachés de correspondencia directa solo se compara una etiqueta).
  - Si ninguna de las etiquetas coincide con la de la dirección emitida por la CPU, el bloque requerido no se encuentra en caché (es decir, se produce un fallo) y es necesario transferirlo desde la memoria principal.
- Una vez identificada la línea que contiene el bloque requerido, se utiliza el **desplazamiento** para acceder a los bytes de la línea que contiene el dato solicitado por la CPU (en el caso de que el dato esté formado por varios bytes, el desplazamiento direcciona el primero de ellos).
- Para determinar si una línea contiene información válida o no, se usa un bit de control denominado **bit de validez** o **bit de válido** (valid bit, V bit).
- Además del bit de válido, las cachés pueden almacenar otros **bits de control**.
- Al calcular el coste de las cachés, hay que incluir el coste debido al almacenamiento de las etiquetas y bits de control.

Ejemplo de identificación de bloque (continuación del ejemplo anterior)

¿Cómo se identifica el bloque 12 ($01100_{(2)}$) en las siguientes cachés?



Las flechas verticales indican comparación de etiquetas en paralelo

Ejemplo de descomposición de direcciones

- Bus de direcciones de 24 bits
- Tamaño de la caché: 8 KBytes (2^{13} Bytes)
- Tamaño de la línea: 8 Bytes (2^3 Bytes)

Núm. de líneas = $2^{13} / 2^3 = 2^{10}$

Caché totalmente asociativa

Etiqueta	Desplazamiento
21 bits	3 bits

Cache de correspondencia directa (2^{10} líneas)

Etiqueta	Índice (línea)	Desplazamiento
11bits	10 bits	3 bits

Caché asociativa por conjuntos de 4 vías

Núm. de conjuntos = 2^{10} líneas / 2^2 vías = 2^8

Etiqueta	Índice (conjunto)	Desplazamiento
13 bits	8 bits	3 bits

Dirección de bloque

Tipos de caché
Políticas de reemplazo (o sustitución)

- **Espacio de reemplazo***:* Es el conjunto de bloques almacenados en caché que pueden ser reemplazados por un determinado bloque de memoria. Depende del tipo de caché:
  - En cachés de correspondencia directa**:** Es el bloque almacenado en la línea donde debe ubicarse el nuevo bloque (no puede hablarse de política de reemplazo).
  - En cachés completamente asociativa**:** Son todos los bloques almacenados en la caché.
  - Asociativa por conjuntos**:** Son todos los bloques almacenados en el conjunto donde debe ubicarse el nuevo bloque.
- Existen diferentes estrategias de reemplazo:
  - **Aleatoria** (Random): Se elige un bloque al azar. Implementación más simple.
  - **FIFO** (First Input First Out): Se sustituye el bloque que más tiempo ha estado en caché.
  - **LRU** (Least Recently Used): Se sustituye el bloque que más tiempo ha estado en la caché sin ser referenciado. Implementación más compleja.
  - **LFU** (Least Frecuently Used): Se sustituye el bloque que menos referencias ha tenido.
- Por cuestiones de coste/eficiencia, es habitual implementar una política que proporciona un *buen* candidato, pero que no garantiza el óptimo. Pseudo-LRU es una implementación simplificada de LRU (que no garantiza el óptimo).
- Los esquemas más utilizados son el aleatorio y el pseudo-LRU.

Tipos de caché
Políticas de reemplazo (o sustitución)

Ejemplo de política LRU en una caché totalmente asociativa de 4 líneas

Línea accedida		3	2	1	0	0	2	3	1	3	0
Línea LRU	0	0	0	0	3	3	3	1	0	0	2

Supongamos que la caché está llena y que, inicialmente, la línea LRU es la 0

Comparativa de frecuencias de fallos en los Alpha (con tamaño de línea de 64 bytes)

	2 vías			**4 vías**			**8 vías**		
	LRU	**Aleat.**	**FIFO**	**LRU**	**Aleat.**	**FIFO**	**LRU**	**Aleat.**	**FIFO**
16KB	**11,41%**	11,73%	11,55%	**11,17%**	11,51%	11,33%	**10,90%**	11,18%	11,04%
64KB	**10,34%**	10,43%	10,39%	10,24%	**10,23%**	10,31%	**9,97%**	10,05%	10,03%
256 KB	9,22%	**9,21%**	9,25%	**9,21%**	**9,21%**	9,25%	**9,21%**	**9,21%**	9,25%

Cuanto más pequeña es la caché y menor asociatividad tiene, más relevante es la política de sustitución

Políticas de escritura: Acierto en escritura

Acierto en escritura: la CPU escribe un dato en una dirección de memoria que corresponde a un bloque que está en caché.

Existen dos posibles implementaciones para los aciertos en escritura:

- **Escritura directa** (Write Through, WT):
  - La palabra se escribe tanto en la línea de caché como en el bloque de memoria principal.
- **Escritura retardada** o **postescritura** (Copy Back, CB):
  - La palabra se escribe solo en caché (es decir, se modifica el bloque que está en caché, pero no la copia que se encuentra en memoria).
  - El bloque modificado en la caché se denomina sucio o modificado.
  - Un bloque sucio se escribe en memoria cuando es reemplazado.
  - Para reducir la frecuencia de escrituras en los reemplazos, se usa un bit de control denominado **bit de sucio (dirty bit, D bit)**, de manera que, si el bloque no está sucio, no se escribe en memoria principal.
  - *Ejemplo: véase "IS-AC-Tema3-EJEMPLO-ACCESO-CACHE.pdf" (diapositivas 14 a 17).*

Políticas de escritura: Acierto en escritura
Ventajas e inconvenientes de la escritura retardada

- **Ventajas:**
  - El tiempo de acceso en escritura es siempre el tiempo de acceso a caché.
  - Múltiples escrituras en un mismo bloque requieren una sola escritura en la memoria principal.
    - Ejemplo: Se ejecuta el siguiente código en una caché de 2MB con líneas de 32 bytes.

```
int v[128]; // cada elemento ocupa 4bytes (v ocupa 128*4/32=16 bloq.)
int i, j; // están en registros
…
for (j=0; j<10; j++)
    for (i=0; i<128; i++)
        v[i] = v[i] + 1;
```

Se realizan 10x128 = 1280 escrituras. En una **caché CB**, cuando las 16 líneas que contienen al vector sean reemplazadas, se realizarán **16 escrituras de un bloque** en memoria (un total de 128 palabras). Sin embargo, en una **caché WT**, se realizarán **1280 escrituras de una palabra** en memoria principal.

  - El tráfico entre caché y memoria principal es menor que en las de escritura directa.
- **Inconvenientes**:
  - Se requiere un bit de control adicional (el bit de sucio).
  - Implementación más compleja: los fallos de lectura pueden provocar escrituras en el nivel inferior y tienen que comprobar el bit de sucio.
  - Durante un periodo de tiempo, la coherencia entre caché y memoria principal no se mantiene.

Políticas de escritura: Acierto en escritura
Ventajas e inconvenientes de la escritura directa

- **Ventajas:**
  - Implementación sencilla (bajo coste hardware).
  - Se mantiene en todo momento la coherencia entre caché y memoria principal.
    - Este aspecto puede ser relevante en el caso de dispositivos de E/S que acceden directamente a memoria (véase el tema 5).
- **Inconvenientes:**
  - El tiempo de acceso en escritura es siempre el tiempo de acceso a memoria principal.
    - Para mitigar este efecto, se suele utilizar un **búfer de escritura** (es decir, una cola que almacena temporalmente los datos mientras que los datos esperan a que sean escritos en memoria principal).
  - Tráfico elevado entre caché y memoria principal (cada escritura en caché provoca una escritura en memoria principal).
    - Este aspecto es crítico en sistemas multiprocesadores en los que múltiples CPU están conectadas a un mismo bus (véase el tema 4).

Políticas de escritura: Fallo en escritura

Fallo en escritura: la CPU escribe un dato en una dirección de memoria que corresponde a un bloque que **no** está en caché.

Existen dos posibles implementaciones para los fallos en escritura:

- **Con ubicación en escritura** (Write Allocate, WA): Cuando se produce un fallo en una operación de escritura, primero se ubica el bloque en caché (es decir, se transfiere de la memoria principal a la caché) y, a continuación, se escribe el dato sobre dicho bloque.
  - *Ejemplo: véase "IS-AC-Tema3-EJEMPLO-ACCESO-CACHE.pdf" (diapositivas 18 a 21).*
- **Sin ubicación en escritura** (No Write Allocate, NWA): Cuando se produce un fallo en una operación de escritura, se escribe el dato en el bloque correspondiente de memoria principal, pero dicho bloque no se ubica en caché.

La política de escritura en caso de fallo es independiente de la política en caso de cierto. Se pueden implementar las 4 posibilidades:

- WT-NWA: Los aciertos en escritura escriben en caché y memoria principal; los fallos, en memoria principal. No hay bloques sucios.
- WT-WA: Todas las escrituras escriben en caché y en memoria principal. No hay bloques sucios.
- CB-NWA: Los aciertos en escritura escriben en caché; los fallos, en memoria principal. Los bloques sucios se escriben en memoria principal cuando son reemplazados
- CB-WA: Todas las escrituras escriben en caché. Solo se escribe en memoria principal cuando un bloque sucio es reemplazado.

Optimización
Cómo mejorar el rendimiento de las cachés

- El objetivo es reducir el tiempo medio de acceso a memoria (AMAT):

> ↓*Tiempo medio de acceso a memoria* = ↓*Tiempo de acierto* +
> ↓*Frecuencia de fallos* × ↓*Penalización por fallo*

- Existen tres formas de reducir el tiempo medio de acceso a memoria:
  - Reducir la frecuencia de fallos de la caché (*miss rate*)
  - Reducir la penalización por fallo (*miss penalty*)
  - Reducir el tiempo de acierto (*hit time*)
- Modelo de las tres C:
  - Nos ayuda a comprender cómo afectan las características de la caché a las tres componentes del tiempo medio de acceso a memoria.
  - Clasifica los fallos en tres tipos:
    - Fallos forzosos (Compulsory misses)
    - Fallos por capacidad (Capacity misses)
    - Fallos por conflicto (Conflict misses)

Cuando se produce un fallo en la caché, el motivo de que el bloque no esté en caché determina el tipo de fallo

Tipos de fallos en las cachés (I): Fallos forzosos

- **Fallos Forzosos (*Compulsory miss*)**
  - **Se producen en el primer acceso a un bloque de memoria**. Este tipo de fallo resulta inevitable puesto que, si es la primera vez que accedemos al bloque, este no puede encontrarse en la caché al no haber sido referenciado anteriormente.
  - También se conocen como *fallos de arranque en frío* o *de primera referencia*.
  - Debido al principio de localidad espacial, **el número de fallos forzosos se reduce aumentando el tamaño de bloque** (véase diapositiva siguiente).
  - El número de fallos forzosos solo depende del tamaño de los bloques (este número se mantiene incluso si tuviéramos una caché con infinitos bloques y vías). Es decir, **la asociatividad y el número de bloques de la caché no influyen en los fallos forzosos**.

Tipos de fallos en las cachés (II): Fallos forzosos: ejemplo

- Supongamos que tenemos un vector de 8 elementos (cada elemento ocupa 1 palabra), y que se acceden a todos sus elementos secuencialmente.
- **El número de fallos forzosos se reduce aumentando el tamaño de bloque** (véase el ejemplo 2).
- Número de fallos forzosos es igual al número de bloques referenciados.

Ejemplo1: Bloques de 2 palabras (2 elementos por bloque). Se producen 4 fallos

B0	v[0]	**Fallo forzoso**
	v[1]	Acierto
B1	v[2]	**Fallo forzoso**
	v[3]	Acierto
B2	v[4]	**Fallo forzoso**
	v[5]	Acierto
B3	v[6]	**Fallo forzoso**
	v[7]	Acierto

Ejemplo 2: Bloques de 4 palabras (4 elementos por bloque). Se producen 2 fallos

B0	v[0]	**Fallo forzoso**
	v[1]	Acierto
	v[2]	Acierto
	v[3]	Acierto
B1	v[4]	**Fallo forzoso**
	v[5]	Acierto
	v[6]	Acierto
	v[7]	Acierto

Bloques de memoria

Elementos del vector

Tipos de fallos en las cachés (III): Fallos por capacidad

- **Fallos por Capacidad (*Capacity miss*)**
  - Ocurren cuando la caché no puede contener todos los bloques necesarios durante la ejecución de un programa.
  - En una caché que esté llena, los bloques en caché son reemplazados por los nuevos bloques que se cargan de memoria. Cuando los bloques reemplazados vuelven a ser referenciados se produce un fallo por capacidad.
  - Estos fallos pueden **reducirse aumentando el tamaño de la caché**

- **Ejemplo:** Suponga un vector que ocupa 8 bloques y una caché con 4 líneas. Si el vector se recorre una vez, solo se producen fallos forzosos. Sin embargo, <u>si se recorre dos veces, se producirán también fallos de capacidad</u> debido a que los 8 bloques no caben en la caché.
  - <u>Primer recorrido</u>: La caché se llena 2 veces produciéndose 8 fallos forzosos.
  - <u>Segundo recorrido</u>: La caché vuelve a llenarse 2 veces produciéndose 8 fallos por capacidad.

Si aumentamos la caché del ejemplo a 8 líneas, los fallos por capacidad desaparecen.

Memoria principal

B0
B1
B2
B3
B4
B5
B6
B7

Caché de mapeado directo o totalmente asociativa

L0	~~B0~~	~~B4~~	**~~B0~~**	**B4**
L1	~~B1~~	~~B5~~	**~~B1~~**	**B5**
L2	~~B2~~	~~B6~~	**~~B2~~**	**B6**
L3	~~B3~~	~~B7~~	**~~B3~~**	**B7**

Fallos Forzosos

Fallos por capacidad

Tipos de fallos en las cachés (IV): Fallos por conflicto

- **Fallos por conflicto (*Conflict miss*) o por colisión**
  - Se producen cuando el núm. de bloques referenciados que se ubican en la misma línea (o el mismo conjunto) es mayor que el grado de asociatividad de la caché (número de líneas en las que puede ubicarse un bloque).
  - Provoca reemplazos en un conjunto o línea aunque el resto de la caché permanezca vacía.
  - **Sólo ocurren en cachés de correspondencia directa o asociativas por conjuntos** (pues en las completamente asociativas sería necesario llenar previamente la caché, por lo que sería fallo por capacidad).
  - Estos fallos pueden **reducirse aumentando la asociatividad** (desaparecen completamente en cachés totalmente asociativas).

- **Ejemplo:** Referencias a 2 bloques distintos en cachés de 4 líneas con diferente asociatividad.

<u>Traza de Accesos</u>
Acceso al Bloque 1
Acceso al Bloque 5
Acceso al Bloque 1
Acceso al Bloque 5

Caché de Mapeado Directo

L0	
L1	~~B1~~ ~~B5~~ ~~B1~~ B5
L2	
L3	

2 Fallos Forzosos
2 Fallos por conflicto

Caché Asociativa de 2 Vías

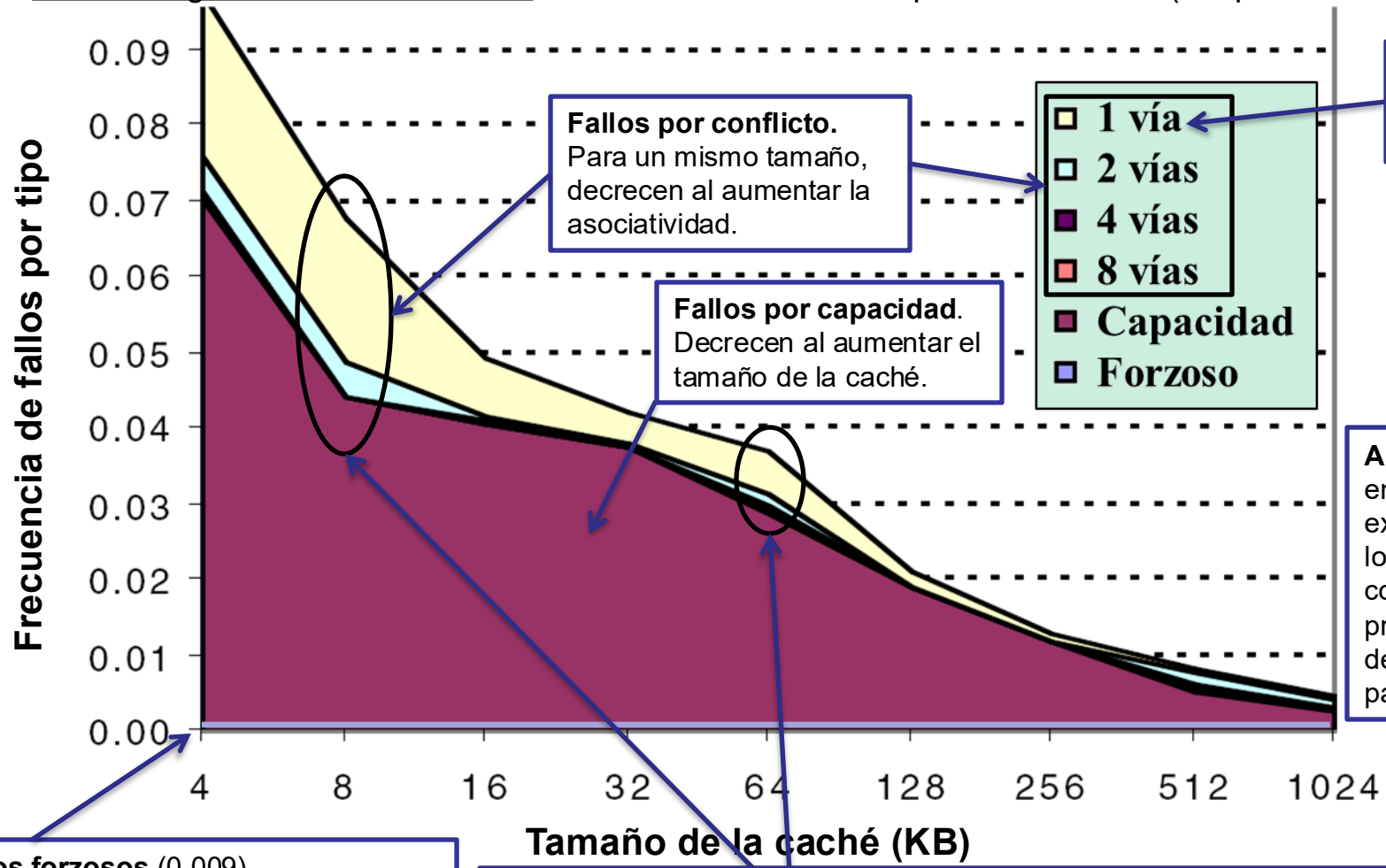
	Vía 0	Vía 1
C0		
C1	B1	B5

2 Fallos Forzosos
No hay fallos por conflicto

No pueden ser fallos por capacidad debido a que solo se están referenciando 2 bloques y en la caché caben 4 bloques

Tipos de fallos en las cachés (V)

Frecuencia total de fallos para diferentes tamaños de caché (el tamaño de bloque no cambia) y diferentes grados de asociatividad. Resultados obtenidos experimentalmente (bloques de 32 bytes).



Fallos por conflicto. Para un mismo tamaño, decrecen al aumentar la asociatividad.

Fallos por capacidad. Decrecen al aumentar el tamaño de la caché.

Caché de mapeado directo

Asociatividad: en los resultados experimentales, los fallos por conflicto prácticamente desaparecen partir de 8 vías.

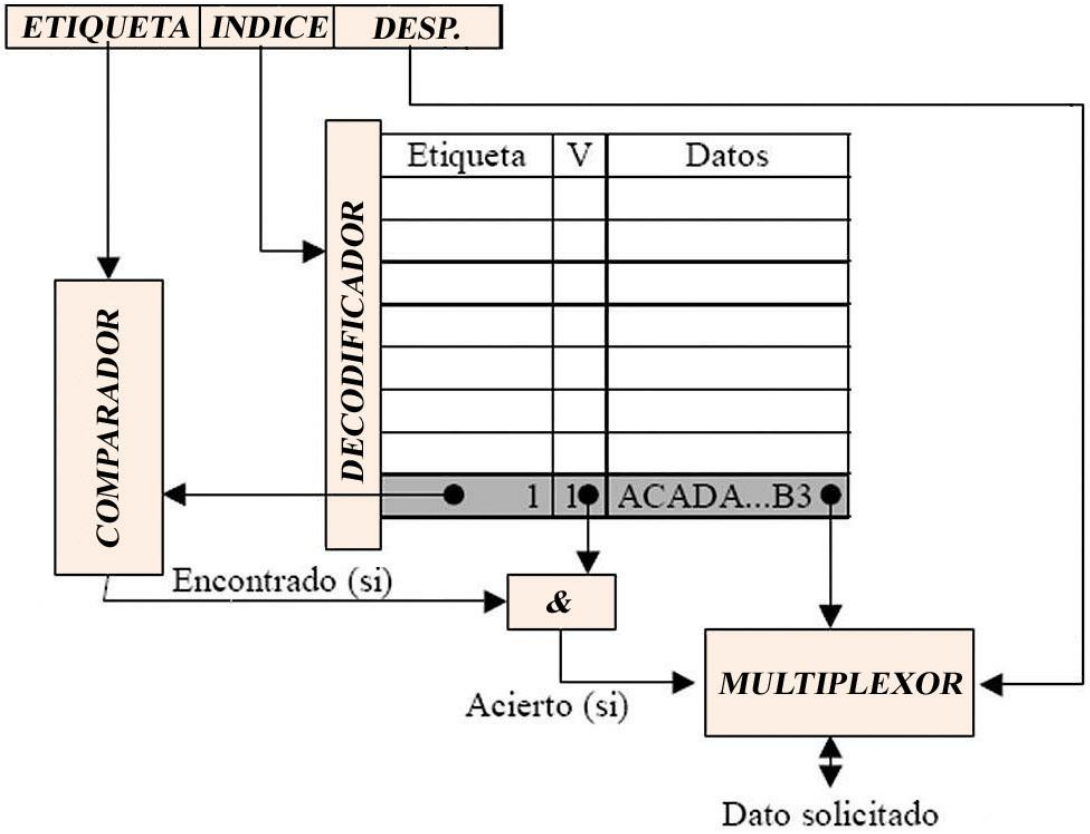
Fallos forzosos (0.009). Independiente del tamaño de la caché (solo dependen del tamaño de bloque)

Incrementar el tamaño de la caché también reduce los **fallos por conflicto**, ya que una caché mayor dispersa las líneas en las que se pueden ubicar los bloques. Ejemplo: en una caché de mapeado directo de 4 líneas, los bloques 0 y 4 se ubican en la misma línea, pero en una caché de 8 líneas se ubican en líneas diferentes.

Implementación
Cachés de correspondencia directa

Inconvenientes:

- **Tasa de fallos alta**: Presentan el mayor número de fallos por conflicto (debido a que tienen el menor grado de asociatividad).



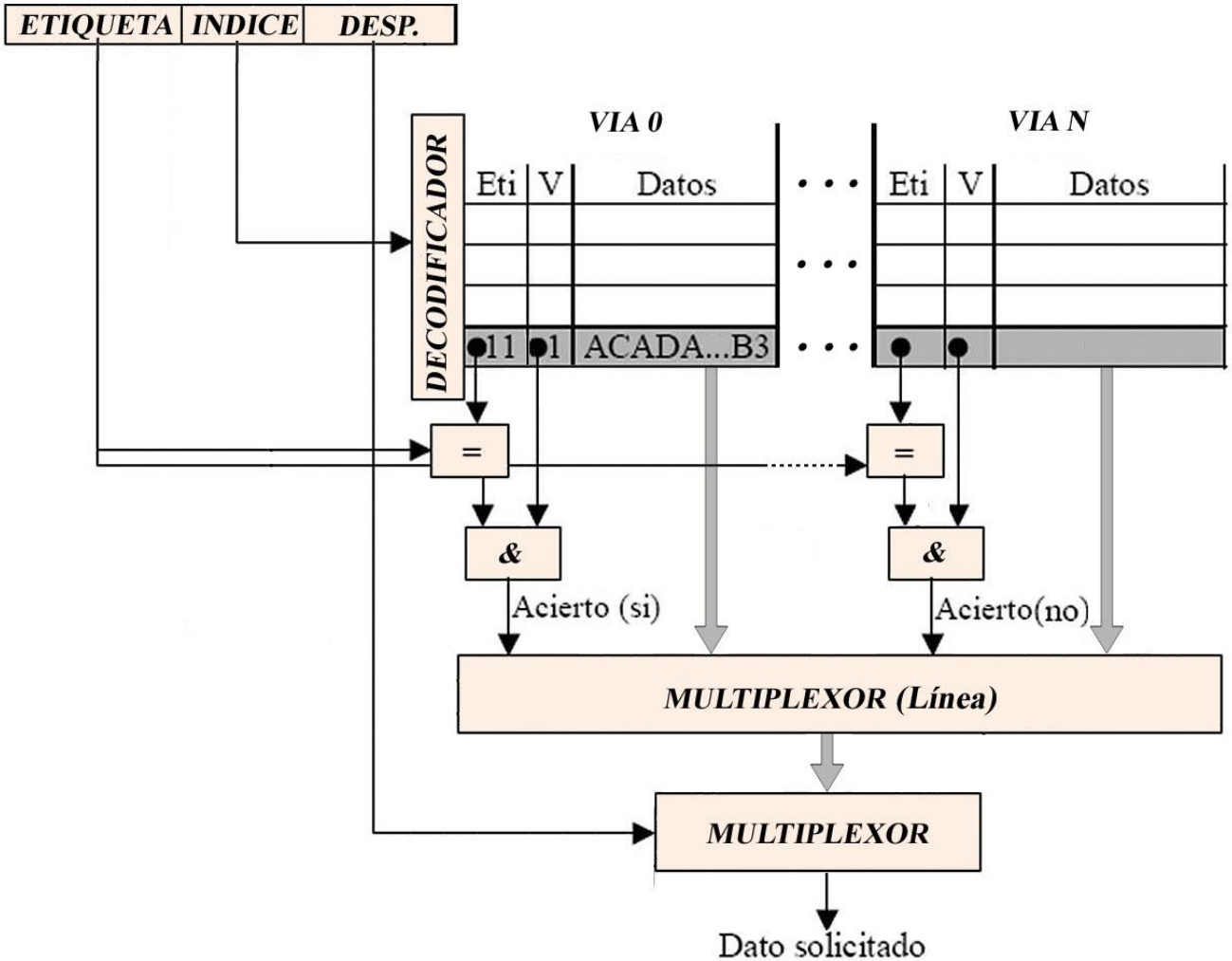
Ventajas:

- **Baja complejidad hardware**: se requiere un único comparador y no se necesita multiplexión de línea (ver cachés asociativas por conjuntos).
- **Identificación rápida**: tiempo de acierto pequeño (no se requiere multiplexión de línea).
- **Poca sobrecarga de memoria**: etiquetas de menor longitud.

Implementación
Cachés asociativas por conjuntos

Ventajas:

- **Al aumentar la asociatividad** (es decir, el número de vías)**, disminuyen los fallos por conflicto**.



Inconvenientes:

Al aumentar la asociatividad:

- **Aumenta el coste hardware** debido a que aumenta el tamaño de las etiquetas, el número de comparadores, el número de entradas del multiplexor de línea, …
- **Aumenta el tiempo de acierto** debido principalmente al multiplexor de línea.

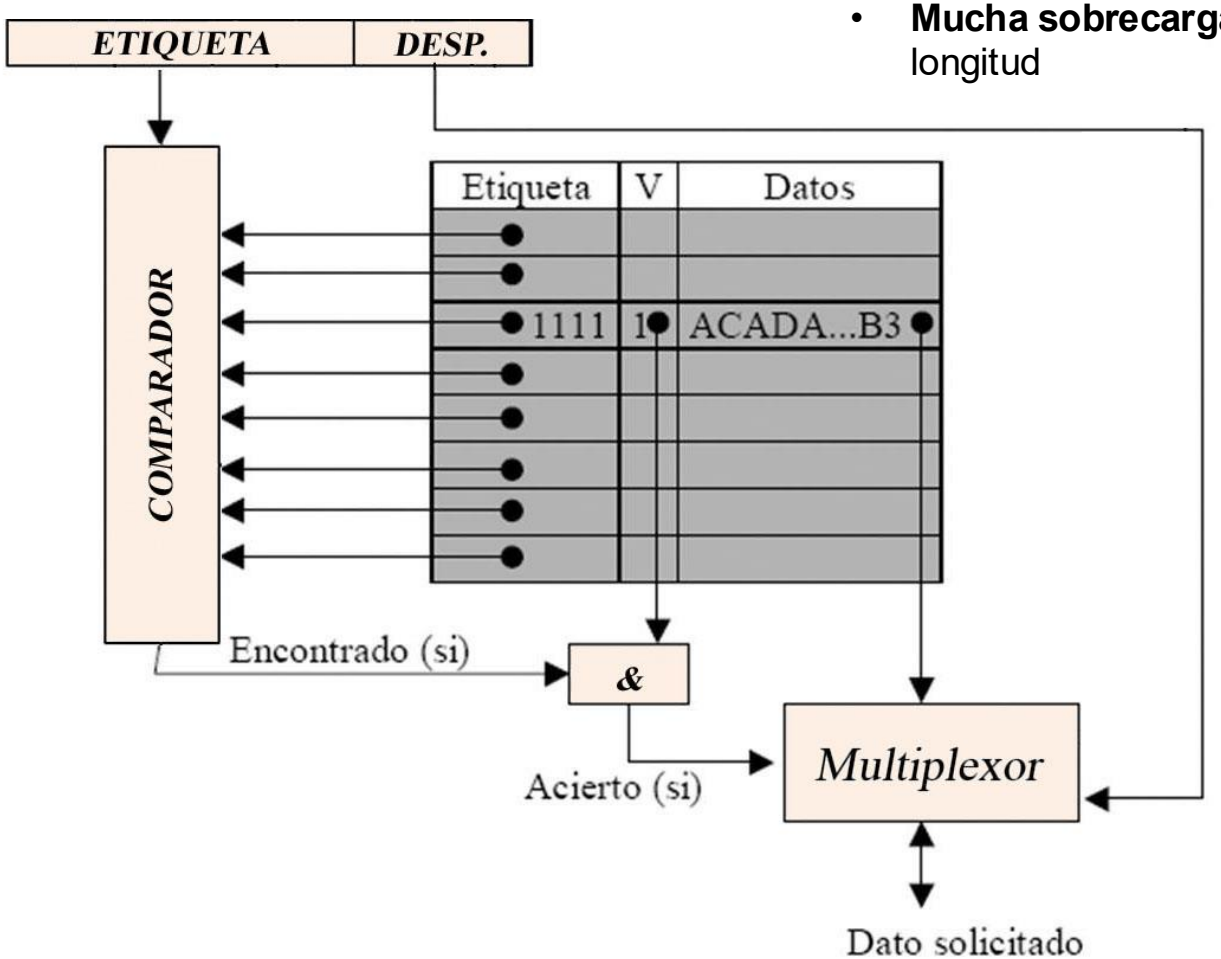
Implementación
Cachés completamente asociativas

Ventajas:

- **Baja tasa de fallos**: solo se producen fallos forzosos y de capacidad.

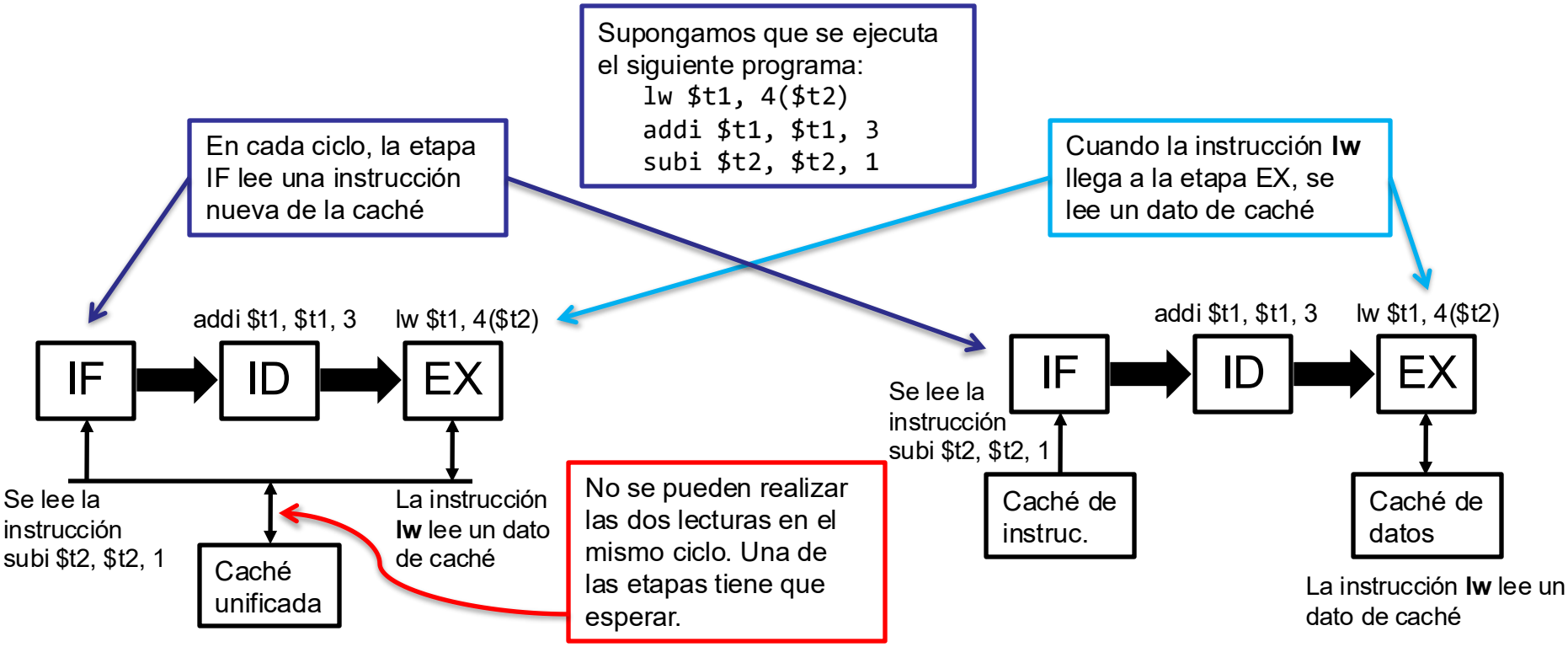
Inconvenientes:

- **Alta complejidad hardware**: se requiere un comparador por cada línea (solo viable para cachés pequeñas).
- **Identificación lenta**: tiempo de acierto grande.
- **Mucha sobrecarga de memoria**: etiquetas de mayor longitud



Caches unificadas vs. Caches separadas (I)

- **Clasificación de las cachés en función del tipo de información que contienen:**
  - **Caches unificadas o mixtas** (*unified or mixed*): Contienen tanto datos como instrucciones
  - **Caches separadas** (*separated*): Existe una caché para datos y otra para instrucciones
- **Cachés y CPU segmentadas:**
  - **Ejemplo:** Supongamos una CPU que divide la ejecución de las instrucciones en tres etapas (**IF**: búsqueda de la instrucción, **ID**: decodificación y **EX**: ejecución).
    - Si la CPU está segmentada, cada etapa ejecuta parte de una instrucción diferente y, en cada ciclo, la instrucción que está en una etapa pasa a la etapa siguiente.



Caches unificadas vs. Caches separadas (II)

- **Ventajas de las cachés separadas:**
  - **Mayor rendimiento: el diseño de cada tipo de caché (de datos y de instrucciones) se puede optimizar por separado (diferentes tamaños de bloques, grados de asociatividad, etc.):**
    - Ejemplo: Las cachés de instrucciones no se escriben, solo se leen (no es necesario implementar las escrituras en caché).
    - Ejemplo: Las cachés de instrucciones tienen menor frecuencia de fallos que las de datos (mayor localidad).
  - **En CPU segmentadas** (diapositiva anterior):
    - **No hay competencia entre la unidad de búsqueda de instrucciones (IF) y la unidad de ejecución (EX).**
    - **Se duplica del ancho de bus entre caché y CPU (puertos separados).**
- **Inconvenientes de las cachés separadas:**
  - **No se equilibra la carga de trabajo automáticamente.**
    - **La separación de instrucciones y datos elimina fallos de conflicto entre bloques de datos y bloques de instrucciones, pero al dividir también se fija el espacio de caché dedicado a cada tipo** (por ejemplo, puede ocurrir que en la caché de datos se produzcan fallos de capacidad mientras que en la caché de instrucciones hay líneas que no se usan; en una caché unificada no ocurriría esto puesto que los datos y las instrucciones comparten la caché).
  - **En general, la tasa de fallos global es algo mayor en las cachés separadas (próxima tranparencia).**

Evolución de memorias caché
Familia de procesadores INTEL

	Procesador	Año	Freq. (MHz)	L1 dato	L1 Instrucción	L2 caché	L3 caché
Sin caché	80386	1985	16-25	-	-	-	-
Unificada	80486	1989	25-100	8KB unificada		-	-
Separadas	Pentium	1993	60-300	8KB	8KB	-	-
	Pentium Pro	1995	150-200	8KB	8KB	256KB-1MB	-
	Pentium II	1997	233-450	16KB	16KB	256-512KB	-
	Pentium III	1999	450-1400	16KB	16KB	256-512KB	-
	Pentium 4	2001	1400-3730	8-16KB	12K- µOps	256KB-2MB	-
	Itanium	2001	800	16KB	16KB	96KB	4MB
	Pentium M	2003	900-2130	32KB	32KB	1-2MB	-
Unificada	Itanium 2	2002	1600	32KB		256KB	6MB
Separadas	Core 2 Duo	2005	1500-2160	32KB/core	32KB/core	2-6MB compartida	-
	Core i7	2008	2660-3200	32KB/core	32KB/core	1MB	8MB compartida

Debido a la ventaja que ofrecen en las CPU segmentadas, la caché L1 se ha mantenido separada en la mayoría de los procesadores de Intel posteriores a 1989.